Brown Sugar Bakery Birthday Club — Architecture & Design

Brown Sugar Bakery — Birthday Club

Architecture & Design Document

Document version: 1.2 **Date:** 2026-04-16 **Status:** Phase 1 in build — ship landing page tonight **Owner:** Robert Dawson

---

1. Executive Summary

The Brown Sugar Bakery Birthday Club is a customer engagement program that turns every household birthday into a recurring cake-purchase opportunity. Customers join through a popup on the Shopify storefront, register their own birthday plus 1–4 family members, and from that point forward an autonomous AI agent handles every customer touchpoint: birthday emails, family-member reminders, weekly specials, and re-engagement nudges.

The program is designed around two strategic principles:

1. **Data agnostic.** All customer data lives in a vendor-neutral database we own. Shopify, the email provider, and any future analytics or marketing tool are interchangeable channels — none of them are the source of truth.
2. **AI-native.** No human is in the loop on day-to-day operations. An AI agent reads business rules, customer state, and current bakery specials, then composes and sends every message itself. Humans edit *the rules*, not the messages.

This document describes the system architecture, data model, workflows, technology choices, and phased build plan.

---

2. Goals and Non-Goals

Goals

- Drive recurring cake orders by activating every birthday in a customer's household, not just the customer's own
- Increase customer lifetime value through automated, personalized engagement
- Make the program operate autonomously — zero ongoing marketing labor
- Keep all customer data in a portable, vendor-neutral store
- Provide logged-in shoppers a visible "Member" experience on every page of the storefront
- Make it easy for shoppers to reorder past purchases (Shop Pay + customer accounts)

Non-Goals (for v1)

- SMS messaging (deferred to v2)
- Loyalty points or tiered membership
- In-store POS integration
- Cake customization wizard
- Multi-location personalization
- A marketer-facing "campaign builder" UI — humans only edit rules in code/config, not campaigns in a dashboard

---

3. Strategic Principles

3.1 Data Agnosticism

Every piece of customer data — birthdays, family members, send history, preferences — lives in **our database**, not in Shopify metafields, not in a marketing tool's CRM. This costs slightly more engineering effort up front and pays back in:

- The ability to switch from Shopify to another commerce platform without losing customer history
- The ability to switch email providers (Resend → Postmark → SendGrid) by changing a single adapter
- The ability to add new channels (SMS, push, direct mail) without re-platforming
- Clean data export for analytics, reporting, or future ML

Implication: The schema is ours to design. Shopify, Resend, and any future tool see *projections* of our data — never the master record.

3.2 AI-Native Operations

Marketing automation is traditionally a human-curated set of “if-this-then-that” workflows built in tools like Klaviyo or Shopify Flow. We are explicitly rejecting that model.

Instead: a Claude-based agent runs on a schedule, queries the database for who needs attention today, reads the current business rules and active specials, and decides per-recipient what to send (or whether to send anything at all). It generates personalized message content and dispatches via the email channel.

Implication: The “campaign” is a prompt and a set of tools, not a flowchart. Adding a new behavior (e.g., “skip customers with an open complaint ticket”) is a single sentence in the prompt or a new tool, not a redesigned flow.

---

4. System Architecture

4.1 Component Diagram

```
+-----------------------------------------------------------------+
|  SHOPIFY STOREFRONT  (channel — replaceable)                    |
|                                                                 |
|  - Liquid theme                                                 |
|  - Birthday Club popup (script injected via theme app extension)  |
|  - Bottom-right "Member" widget for logged-in customers         |
|  - New customer accounts (passwordless OTP, optional at checkout) |
|  - Shop Pay for saved-payment / fast reorder                    |
|                                                                 |
|        |  reads window.Shopify.customer for personalization     |
|        |  POSTs signup data to our API                          |
|        v                                                        |
+--------|--------------------------------------------------------+
         |
         |  HTTPS (CORS)
         |
+--------v--------------------------------------------------------+
|  OUR API + DATABASE  (source of truth — owned)                  |
|                                                                 |
|  Postgres (Supabase) — all customer + program data              |
|  Lightweight HTTP API (Cloudflare Workers)                      |
|    POST /signup           — create or update member             |
|    GET  /membership/:id   — fetch for the storefront widget     |
|    POST /family           — add/edit family members             |
|  Shopify webhook receiver:                                      |
|    /webhooks/orders-create                                      |
|    /webhooks/customers-update                                   |
|                                                                 |
|        ^                                                        |
|        |  reads / writes                                        |
+--------|--------------------------------------------------------+
         |
+--------|--------------------------------------------------------+
|  AI AGENT  (the marketer — autonomous)                          |
|                                                                 |
|  Claude Agent SDK, scheduled (daily 09:00 + weekly Sunday 10:00)  |
|  Tools the agent can call:                                      |
|    - query_birthdays(window_days)                               |
|    - get_active_specials()                                      |
|    - get_business_rules()                                       |
|    - get_customer_history(customer_id)                          |
|    - create_shopify_discount(customer, percent, expires)        |
|    - send_email(to, template, variables)                        |
|    - log_send(customer_id, type, status)                        |
|                                                                 |
|  Reports daily summary to ops email; never sends without rules  |
+--------|--------------------------------------------------------+
         |
+--------v--------------------------------------------------------+
|  EXTERNAL CHANNELS  (transports — replaceable)                  |
```

```
|                                                                  |
|  Resend         — transactional + marketing email                |
|  Shopify Admin  — discount-code creation only                    |
|  (Future) Twilio — SMS                                           |
|  (Future) Postmark — email failover                              |
+------------------------------------------------------------------+
```

4.2 Layer responsibilities

Layer	Responsibility	What it does NOT do
Shopify storefront	Render commerce UI, accept popup signups, host customer login	Hold birthday data, decide what messages to send
Our API	Read/write source-of-truth data, validate input, publish events	Render UI, compose messages
Our database	Persist customers, family members, sends, rules, specials	Anything else
AI agent	Read state, decide who/what/when, generate messages, dispatch	Hold long-term state (it queries every run)
Email channel	Deliver bytes to inboxes	Make any decision about content or recipients
Shopify Admin API	Generate per-recipient discount codes	Hold birthday data

4.3 Why this shape

- **Separation of channel from brain.** Channels (Shopify, Resend) are commodity; the brain (agent + DB) is the asset.
- **Future Shopify replacement is one component swap**, not a rebuild.
- **Email provider swap is a 30-line adapter change**, not a marketing-data migration.
- **Adding new channels (SMS, push) is additive**, not destructive.

---

5. Data Model

The database is Postgres. Schema is intentionally small — five tables cover v1.

5.1 Tables

`members` — one row per Birthday Club member (the human who joined)

Column	Type	Notes
id	uuid PK	
email	citext UNIQUE	Login key, also Shopify customer match key
first_name	text	
last_name	text	
birthday	date	Month/day matter; year used for age tracking
shopify_customer_id	bigint NULL	Populated once we see them in Shopify
joined_at	timestamptz	
status	text	active / unsubscribed / bounced
timezone	text	Default America/Chicago

`family_members` — 0..N per member

Column	Type	Notes
id	uuid PK	
member_id	uuid FK → members.id	
first_name	text	
last_name	text	NULL allowed
relationship	text	spouse / child / parent / sibling / friend / other
birthday	date	

`specials` — bakery promotions the agent can offer

Column	Type	Notes
id	uuid PK	
name	text	e.g. “Spring strawberry shortcake”
description	text	Used by agent in message body
discount_percent	int	0–100
valid_from	date	
valid_to	date	
applicable_to	text	“self” / “family” / “any”
active	bool	
product_handle	text NULL	Optional Shopify product link

`message_log` — every send the agent has ever made

Column	Type	Notes
id	uuid PK	
member_id	uuid FK	
about_person	text	“self” or family_member.id
template	text	Template filename
subject	text	What was sent
discount_code	text NULL	If a code was generated
sent_at	timestamptz	
channel	text	“email” / future “sms”
provider_message_id	text	Resend’s id, for tracking
status	text	sent / delivered / opened / clicked / bounced

`business_rules` — versioned config the agent reads

Column	Type	Notes
id	uuid PK	
key	text	e.g. “self_birthday_discount_percent”
value	jsonb	
effective_from	date	
notes	text	Why this rule exists

5.2 Why not Shopify metafields

Shopify metafields would force this schema into a vendor-specific shape, make exports painful, and tie the data lifecycle to Shopify’s product roadmap. Keeping it in our own Postgres preserves portability. The single field we *do* push back to Shopify is the customer tag `birthday-club`, used purely so Liquid can render the membership widget without an extra API call.

---

6. Key Workflows

6.1 Signup workflow

```
Visitor on storefront
  │
  ├─► Clicks "Join the Birthday Club" floating button
  │
  ├─► Modal opens (rendered by injected JS)
  │
  ├─► Fills name / email / birthday
  │
  ├─► Optionally adds 1–4 family members
  │
  ├─► Submits → POST /signup to our API
  │       │
  │       ├─► API validates input
  │       ├─► API upserts members row
  │       ├─► API inserts family_members rows
  │       ├─► API enqueues "tag in Shopify" job
  │       └─► API returns success
  │
  ├─► Modal shows confetti + welcome message
  │
  └─► Async: background worker tags the Shopify
            customer with "birthday-club" so the
            storefront widget recognizes them next visit
```

6.2 Daily birthday agent run

```
09:00 America/Chicago — Cron Trigger fires
  │
  ├─► Agent process starts (Claude Agent SDK)
  │
  ├─► Agent prompt:
  │     "It's [date]. Find everyone whose birthday
  │      (their own or a family member's) is in the
  │      next 7 days and who hasn't been messaged
  │      yet for that occasion. Apply business rules.
  │      Pick the most relevant active special.
  │      Compose a personalized message. Generate a
  │      unique discount code. Send via email. Log."
  │
  ├─► Agent calls query_birthdays(window=7)
  │     → returns list of [member, target_person, days_until]
  │
  ├─► Agent calls get_active_specials()
  │     → returns currently-running promos
  │
  ├─► Agent calls get_business_rules()
  │     → returns config (discount levels, send windows, etc.)
  │
  ├─► For each recipient:
  │     ├─► Agent calls get_customer_history(member_id)
  │     │     → recent orders, last contact, opens/clicks
  │     │
  │     ├─► Agent reasons: skip? send now? which special?
  │     │
  │     ├─► If sending:
  │     │     ├─► Agent calls create_shopify_discount(...)
  │     │     │     → returns unique code
  │     │     ├─► Agent calls send_email(to, template, vars)
  │     │     │     → Resend delivers
  │     │     └─► Agent calls log_send(...)
  │     │
  │     └─► Else: log decision and reasoning
  │
  └─► Agent posts daily summary to ops@brownsugarbakery.com:
      "Today: 12 birthdays found, 11 emails sent, 1 skipped
       (recent order). 3 family-member birthdays this week.
       2 unsubscribes processed. Provider health: green."
```

6.3 Logged-in member visits storefront

```
Customer visits brownsugarbakery.com
  │
  ├─► Liquid theme renders page
  │
  ├─► theme.liquid checks:
  │     {% if customer and customer.tags contains 'birthday-club' %}
  │
  ├─► If yes: bottom-right widget renders
  │     ├─► Shows "Member" badge
  │     ├─► Shows days-until-next-birthday (theirs or family)
  │     └─► Click → opens member dashboard modal
  │
  ├─► Dashboard modal calls GET /membership/:email
  │     → our API returns birthday + family list + recent sends
  │
  └─► Customer can edit family, update birthday, unsubscribe
```

6.4 Shopify event sync (inbound)

```
Customer places order on Shopify
  │
  └─► Shopify webhook orders/create POSTs to our API
        │
        ├─► API verifies HMAC signature
        ├─► API matches order's customer to members.email
        ├─► API records order metadata (date, value, items)
        │     into a lightweight orders table (for agent context)
        └─► API responds 200
```

---

7. Technology Stack

Layer	Choice	Why this, not the alternative
Database	**Supabase (Postgres)**	Postgres = portable; Supabase = zero-ops + free tier; data is `pg_dump`-exportable to anywhere
API runtime	**Cloudflare Workers**	Free tier covers expected volume, no cold starts, global edge, simple deploys
Agent runtime	**Cloudflare Workers + Cron Triggers**	Same env as API, cron is a one-liner config
Agent framework	**Claude Agent SDK**	Built for headless tool-using agents; first-class Anthropic support
Email transport	**Resend**	Modern API, 3k/mo free, clean DX, easy template management
Templates	**Plain HTML files with `{{variable}}` substitution**	No build step, no JSX runtime, easy to edit, version-controlled
Storefront integration	**Shopify Theme App Extension (script injection)**	Survives theme updates, deployable independently of theme code
Customer accounts	**Shopify New Customer Accounts** (passwordless OTP)	Classic accounts deprecated Feb 2026; OTP reduces password friction
Hosting (storefront)	**Shopify**	The commerce engine — kept narrowly scoped to its job
Image hosting	**Supabase Storage** or Shopify Files	Brand assets referenced by URL from email templates
Source control	**GitHub**	Standard
Monitoring	**Cloudflare Workers logs + Resend dashboard**	Free, sufficient for v1

Cost projection (monthly, at expected v1 volume)

Item	Cost
Supabase (Free tier: 500MB DB, 5GB bandwidth)	$0
Cloudflare Workers (Free: 100k req/day)	$0
Resend (Free: 3k emails/month)	$0
Anthropic API (agent runs ~30/day, modest token usage)	~$15–40
Total for v1	**~$15–40/month**

At ~5,000 members + ~10,000 emails/month: ~$80–150/month. Scales linearly with membership.

---

8. Component Designs

8.1 Storefront integration

A single Shopify app with one job: inject `<script src="https://api.brownsugarbakery.com/storefront.js" defer>` into every storefront page via a theme app extension.

`storefront.js` does: 1. Renders the floating “Join” button (CSS-isolated via shadow DOM to avoid theme conflicts) 2. Reads `window.Shopify.customer` to detect login state 3. If logged-in member, renders the bottom-right widget instead of the join button 4. Opens the modal on click; modal posts to our API 5. On member-dashboard view, fetches from our API

Why a script (not Liquid edits): theme updates don’t break us, deploys are independent of merchandising team’s theme work, one codebase serves all themes the bakery may use.

8.2 API service

Cloudflare Worker with these routes:

Method	Path	Purpose
POST	/signup	Create or update a member + family
GET	/membership/:email	Fetch member view for storefront widget
POST	/family	Add or edit family member
DELETE	/family/:id	Remove family member
POST	/unsubscribe/:token	Honor unsubscribe link
POST	/webhooks/shopify/orders-create	Inbound order event
POST	/webhooks/shopify/customers-update	Inbound customer update
GET	/storefront.js	Serve the storefront bundle

All routes: - Validate input with a Zod schema - Use Supabase service-role key (kept in Workers secret env) - Verify Shopify webhook HMAC where applicable - Apply rate limiting (Cloudflare's built-in)

8.3 The AI agent

Runtime: A scheduled Worker that boots, runs the Claude Agent SDK with a system prompt and a tool set, waits for the agent to finish, then exits.

System prompt (sketch):

> *You are the marketing operator for Brown Sugar Bakery's Birthday Club. You run once daily and once weekly. Your job is to find members whose birthdays (their own or a family member's) are within the upcoming send window, apply business rules, choose the most appropriate active bakery special, generate a personalized message, create a unique discount code, and send the email.*
>
> *Operating principles: - Never message anyone twice for the same occasion. - Skip members who have placed an order in the last 48 hours. - Skip members who are unsubscribed or have bounced. - Honor each member's preferred timezone for timing. - Tone: warm, casual, Chicago-bakery friendly. Never corporate. - Always sign emails from the Brown Sugar Bakery team. - Stop and report any unusual conditions (no active specials, provider errors, etc.).*
>
> *When you finish, summarize what you did, what you skipped (and why), and any issues for the operator.*

Tools (each is a small TypeScript function the agent can call):

```
query_birthdays(window_days: int) → BirthdayCandidate[]
get_active_specials() → Special[]
get_business_rules() → Rules
get_customer_history(member_id: uuid) → History
create_shopify_discount(member_id, percent, expires_at) → DiscountCode
send_email(to, template_name, variables) → MessageId
log_send(member_id, occasion, template, code, message_id) → void
```

Why an LLM agent instead of a cron + templates:

Capability	Cron + templates	LLM agent
Pick best special per recipient	Hand-coded heuristics	Reads context, decides
Personalize tone per recipient	Static templates	Adapts to context
Skip on edge cases (recent order, complaint)	Add condition to flow	Add sentence to prompt
Add new behavior (e.g., “mention their child by name”)	Code change + deploy	Edit prompt
Handle weird input gracefully	Crashes or skips	Reasons about it

The cost of putting an LLM in the loop is small (~$15–40/month) and the operational leverage is large.

8.4 Email template system

Templates live in the repo at `templates/*.html`. Each template is plain HTML with `{{variable}}` placeholders. A shared `_layout.html` provides brand wrapping (logo, colors, footer, unsubscribe link).

Initial template set:

File	Trigger
`welcome.html`	New member signs up
`birthday_self.html`	Member’s own birthday
`birthday_family.html`	A family member’s birthday
`weekly_special.html`	Weekly Sunday push (selective)
`reactivation.html`	Member hasn’t engaged in 90 days

Variables passed in by the agent. Images referenced by URL from Supabase Storage. Inlined CSS for email-client compatibility.

Render path: agent calls `send_email(to, "birthday_self", { firstName, discountCode, expiresOn })` → tool loads template, substitutes vars, posts to Resend `/emails` endpoint with rendered HTML + plaintext fallback.

8.5 Social landing page

Separate from the storefront popup, a **dedicated landing page** at `brownsugarbakery.com/pages/birthday-club` is the destination for social media posts and paid ads.

Why separate from the popup:

- Social platforms (Instagram bio link, Facebook ads, TikTok link-in-bio) need a single shareable URL
- Paid ads need a destination URL with conversion tracking
- A full-page format outperforms a popup for cold traffic (visitors who came specifically to learn about the program)

Implementation:

- Lives as a Shopify Page entity with handle `birthday-club` → URL `/pages/birthday-club`
- Custom Liquid template `templates/page.birthday-club.liquid` rendered by the theme
- Full-page hero layout: brand banner, program benefits, testimonials/photos, FAQ, inline signup form (NOT a popup on this page)
- Inline form posts to the same `POST /signup` API endpoint as the popup — one signup pipeline, one data model
- Open Graph and Twitter Card meta tags for rich social previews
- Captures UTM parameters (`?utm_source=instagram&utm_campaign=summer`) into a `signup_source` field on the `members` table for attribution
- Mobile-first design (most social traffic is mobile)
- Same brand styling as the popup so both feel cohesive

Page sections (top to bottom):

1. Hero — logo, headline ("Because Everyone Has a Birthday"), single primary CTA scrolling to form
2. Benefits — same four benefits from the popup, presented as a richer card grid
3. How it works — 3-step explainer (Join → Tell us about your family → Get a sweet message before each birthday)
4. Social proof — bakery photos, optional testimonials
5. Inline signup form — name, email, birthday, family members (same data shape as popup)
6. FAQ — discount details, what to expect, how to unsubscribe
7. Footer — same as storefront

Routing the popup vs the landing page:

- Popup remains the default capture surface across the storefront

- Landing page is the explicit destination for off-site traffic
- Both feed the same database and trigger the same welcome email — no parallel pipelines

8.6 Shopify integration surface

Deliberately minimal:

What	Direction	Mechanism
Inject storefront popup	Outbound (us → SF)	Theme app extension
Read login state	In-browser	`window.Shopify.customer`
Tag member as `birthday-club`	Outbound (us → SF)	Admin GraphQL `customerUpdate`
Receive order events	Inbound (SF → us)	Webhook
Receive customer updates	Inbound (SF → us)	Webhook
Generate discount codes	Outbound (us → SF)	Admin GraphQL `discountCodeBasicCreate`

Notably absent: customer metafield writes, metaobject definitions, Customer Account UI Extensions, Shopify Email, Shopify Flow. We avoid them deliberately to keep Shopify as a thin channel.

---

9. Security & Privacy

- **PII at rest:** Encrypted by Supabase (AES-256). Only the API service role can read.
- **PII in transit:** HTTPS only, enforced by Cloudflare.
- **Secrets:** Stored in Cloudflare Workers encrypted env vars. Never committed.
- **Unsubscribe:** Every email includes a one-click unsubscribe link with a signed token. Honored within seconds.
- **Bounce handling:** Resend webhook updates `members.status`; agent skips bounced addresses.
- **GDPR / CCPA:** Member can request data export or deletion via support email; deletion cascades family_members and anonymizes message_log.
- **Webhook verification:** All Shopify webhooks verified via HMAC against shared secret before processing.
- **Rate limiting:** Cloudflare built-in on all public routes.
- **Agent guardrails:** Agent is read-only on customer data except `message_log` writes. It cannot modify members' contact info, change their family, or alter business rules.

10. Phased Build Plan

The project ships in **two phases**. Phase 1 launches the campaign and starts collecting member data. Phase 2 turns on autonomous AI agent operations on top of that data.

Phase 1 — Capture (LAUNCH NOW)

Goal: Get the landing page live on the Shopify storefront and start collecting Birthday Club signups immediately.

Scope:

- Dedicated landing page at `brownsugarbakery.com/pages/birthday-club`
- Single self-contained Liquid template with inline form, brand styling, and Supabase signup logic
- Open Graph + Twitter Card meta tags for social sharing previews
- UTM parameter capture (`?utm_source=instagram` etc.) to track campaign attribution
- Form posts directly to the existing Supabase database (`members` + `family_members` tables already exist)
- Existing welcome-email edge function continues to fire on signup
- Mobile-first responsive layout

What is reused from the prototype:

- Supabase project, anon key, schema (already provisioned)
- The form fields, validation, and confetti polish (already built in `app.js` / `styles.css`)
- The brand assets (logo, color palette, fonts)
- The welcome email edge function (`send-welcome-email`)

What is NOT in Phase 1:

- AI agent — manual review of signups in Supabase Studio for now
- Birthday email automation — collected data sits, no recurring sends yet
- Storefront popup on every page — landing page only for now
- Logged-in member widget — deferred
- Cloudflare Workers API, Shopify app, theme app extension — deferred
- New customer-account integration — deferred

Deployment surface:

- One Shopify Page entity created in admin (handle: `birthday-club`)
- One Liquid template uploaded to the theme (`templates/page.birthday-club.liquid`)
- One image uploaded to Shopify Files (the bakery logo)
- DNS / domain unchanged

Success criteria:

- Page live and indexable at `brownsugarbakery.com/pages/birthday-club`
- Signup form submits successfully on mobile and desktop
- New rows appear in Supabase `members` table within seconds of submission
- UTM parameters captured on signups originating from social posts
- Page loads in under 2 seconds on 4G mobile

Phase 2 — Activation (LATER)

Goal: Turn on autonomous AI-driven engagement on top of the data collected in Phase 1.

Scope:

- Cloudflare Workers API service replacing direct Supabase writes (so future apps share one signup pipeline)
- AI agent (Claude Agent SDK) running on schedule — daily birthday sends, weekly specials, reactivation
- Email template library (5 templates: welcome, birthday_self, birthday_family, weekly_special, reactivation)
- Resend integration with verified sender domain
- Storefront popup deployed via theme app extension (visible on every page, not just the landing page)
- Logged-in member widget (bottom-right "Member" badge with days-until-birthday)
- Customer Account UI Extension for "My Birthday Club" inside Shopify accounts
- Customer-tag sync (us → Shopify) so storefront recognizes members
- Inbound webhooks (orders/create, customers/update)
- Business-rules config in repo (`config/business_rules.yaml`)
- Bounce handling, unsubscribe flow, GDPR/CCPA data export
- Daily ops summary email to `ops@brownsugarbakery.com`
- 1-week dry-run period before first live send

Sequenced sub-phases inside Phase 2:

1. **Foundations:** API service, business-rules config, email template library

2. **Member experience:** popup, logged-in widget, account UI extension
3. **The agent:** scheduled runner, tools, prompt, dry-run period
4. **Go-live:** monitoring, hardening, ops handoff

Why this split

- **Capturing data is urgent.** Every day without data is a day of lost engagement opportunity. Phase 1 is hours of work, not weeks.
- **The AI agent is the *answer*, not a prerequisite.** It can be built against real data accumulated during Phase 1, which improves prompt tuning and rule design.
- **Risk is minimized.** Phase 1 reuses already-working code; Phase 2 is greenfield engineering that benefits from real-world data to validate against.
- **Marketing can launch the campaign now** — paid ads, Instagram link-in-bio, in-store QR codes — while the engineering build continues in parallel.

---

11. Locked Decisions

All foundational decisions are locked as of 2026-04-16. The agent and supporting code are built against these values.

#	Decision	Locked value
1	Sender address	**birthdayclub@brownsugarbakery.com**
2	Member's own birthday discount	**30%**
3	Family member birthday discount	**20%** (matches the existing popup promise)
4	Send timing	**7 days before** the birthday
5	Discount code expiration	**14 days** from issue
6	Allow guest signup (no Shopify account required)	**Yes** — silently create the Shopify customer record; claim-account flow at first order
7	Weekly special email	**Send only to members who have no birthday occasion in the coming week**
8	Ops summary recipient	**Shared inbox: ops@brownsugarbakery.com**
9	Dry-run period before live sends	**1 week** of daily summaries
10	Business-rules ownership going forward	**Engineering edits a config file in the repo via PR** (`config/business_rules.yaml`); changes are versioned, reviewable, rollback-able

11.1 Implications of these decisions

- **Discount rules** in `config/business_rules.yaml`:

  ```yaml
  discount_self_birthday_percent: 30
  discount_family_birthday_percent: 20
  discount_validity_days: 14
  send_window_days: 7
  ```

- **Account creation** is implicit at signup. The signup endpoint calls Shopify Admin `customerCreate` with the email; Shopify's New-customer-accounts OTP claim flow handles credential setup later. No password is required at signup.
- **Weekly special exclusion** requires the agent to query both `members` and `message_log` for the upcoming 7-day window before sending the Sunday weekly push.

- **Config-as-code** for business rules means rule changes go through GitHub PRs; the agent reads from a YAML file at runtime (cached per run).
- **Shared ops inbox** must be created and its address verified in Resend so daily summaries deliver reliably.

---

12. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Agent sends inappropriate content	Low	High	System prompt is conservative; dry-run period; daily summary review
Email deliverability issues	Medium	Medium	Proper SPF/DKIM/DMARC at setup; warm up sending domain gradually
Shopify API changes break sync	Medium	Low	Pin API version; webhook events are stable surface
Supabase outage	Low	High	Daily DB backup to S3; agent fails gracefully and reports
Resend outage	Low	Medium	Adapter pattern allows fallback to Postmark in <1 day
Anthropic API outage	Low	Low	Agent retries; missed day's birthdays caught by next run within window
Member data export request	Certain over time	Low	GDPR/CCPA flow built into Phase 5
Bounce/complaint rate damages sender reputation	Medium	High	Unsubscribe respected immediately; bounced addresses skipped

---

13. Glossary

- **Member** — A customer who has joined the Birthday Club.
- **Family member** — A person registered by a member, not a Shopify customer themselves.
- **Occasion** — A specific birthday event (member's own, or a family member's). Each occasion gets at most one send.
- **Special** — A bakery promotion the agent may attach to a message.
- **Send window** — The number of days before an occasion when the agent may send (default 7).
- **Dry-run** — The agent reasons and logs its decisions but does not actually send emails or create discounts.

Appendix A — Example Agent Daily Summary

```
Subject: Birthday Club daily run — 2026-04-23

Run started: 09:00:03 CDT
Run ended:   09:00:47 CDT

CANDIDATES FOUND: 14
  - 4 self-birthdays today
  - 3 self-birthdays in next 7 days
  - 7 family-member birthdays in next 7 days

ACTIONS TAKEN: 12
  - 12 birthday emails sent (template: birthday_self x4,
    birthday_family x7, welcome x1)
  - 12 unique discount codes created in Shopify

SKIPPED: 2
  - jane@example.com — placed order yesterday
    (rule: skip-if-recent-order)
  - bob@example.com — bounce status
    (rule: skip-if-bounced)

ACTIVE SPECIAL CHOSEN MOST OFTEN:
  "Spring strawberry shortcake" (8 of 12 sends)

ISSUES: none

NEXT RUN: 2026-04-24 09:00 CDT
```

Appendix B — Example Template Snippet

```
<!-- templates/birthday_self.html -->
{{> _layout_open}}

<h1 style="font-family: 'Tenor Sans', serif; color: #6B3410;">
  Happy Birthday, {{firstName}}!
</h1>

<img src="https://cdn.brownsugarbakery.com/email/birthday-cake.jpg"
     alt="Birthday cake"
     style="width: 100%; border-radius: 12px; margin: 24px 0;">

<p>It's your day, and we're celebrating you.</p>

<p>Use code <strong style="background:#FFE4C4; padding:4px 8px;
   border-radius:4px;">{{discountCode}}</strong> for
   <strong>{{discountPercent}}% off</strong> anything in the bakery.</p>

<p>Code expires {{expiresOn}}. Order online or stop by.</p>

<p style="margin-top: 32px;">Life is sweet,<br>
The Brown Sugar Bakery team</p>

{{> _layout_close}}
```

Appendix C — Example Schema DDL

```sql
create extension if not exists "citext";

create table members (
  id uuid primary key default gen_random_uuid(),
  email citext unique not null,
  first_name text not null,
  last_name text not null,
  birthday date not null,
  shopify_customer_id bigint,
  joined_at timestamptz not null default now(),
  status text not null default 'active'
    check (status in ('active','unsubscribed','bounced')),
  timezone text not null default 'America/Chicago'
);

create table family_members (
  id uuid primary key default gen_random_uuid(),
  member_id uuid not null references members(id) on delete cascade,
  first_name text not null,
  last_name text,
  relationship text not null,
  birthday date not null
);

create table specials (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  description text not null,
  discount_percent int not null check (discount_percent between 0 and 100),
  valid_from date not null,
  valid_to date not null,
  applicable_to text not null
    check (applicable_to in ('self','family','any')),
  active boolean not null default true,
  product_handle text
);

create table message_log (
  id uuid primary key default gen_random_uuid(),
  member_id uuid not null references members(id) on delete cascade,
  about_person text not null,
  template text not null,
  subject text not null,
  discount_code text,
  sent_at timestamptz not null default now(),
  channel text not null default 'email',
  provider_message_id text,
  status text not null default 'sent'
);

create index message_log_member_occasion_idx
  on message_log(member_id, about_person, sent_at);
```

```sql
create table business_rules (
  id uuid primary key default gen_random_uuid(),
  key text not null,
  value jsonb not null,
  effective_from date not null default current_date,
  notes text
);
```

End of document.